



파머완 10주차

▼ Chapter 6. 차원 축소

▼ 1. 차원 축소의 개요

차원 축소: 매우 많은 피처로 구성된 다차원 데이터 세트의 차원을 축소해 새로운 차원의 데이터 세트를 생성하는 것

차원 증가 → 데이터 포인트 간의 거리가 기하급수적으로 멀어져 희소한 구조를 가지게 됨

수백 개 이상의 피처로 구성된 데이터 세트

- 적은 차원에서 학습된 모델보다 예측 신뢰도 떨어짐
- 개별 피처 간 상관관계 높을 가능성이 커서 선형 모델에서 예측 성능 저하

→ 차원 축소 필요

피처 선택: 특정 피처에 종속성이 강한 불필요한 피처는 아예 제거하고, 데이터의 특징을 잘 나타내는 주요 피처만 선택하는 것

피처 추출: 기존 피처를 저차원의 중요 피처로 압축해서 추출하는 것

- 기존 피처가 인지하기 어려웠던 잠재적인 요소 추출

차원 축소의 사용

- 많은 픽셀로 이루어진 이미지 데이터에서 잠재된 특성을 피처로 도출해 함축적 형태의 이미지 변환과 압축 수행 가능

→ 과적합 영향력이 작아져 예측 성능 증가

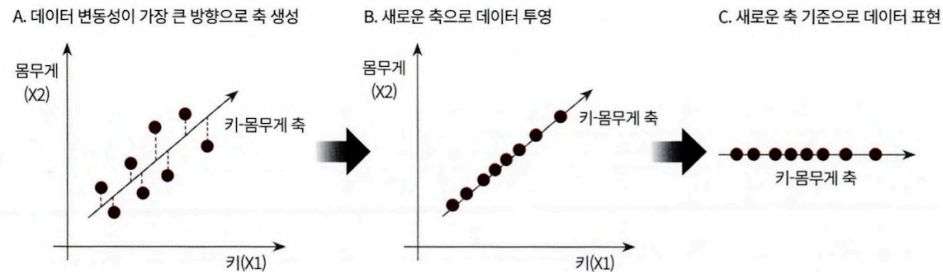
- 텍스트 문서의 숨겨진 의미 추출

시맨틱 토픽 모델링

▼ 2. PCA (Principal Component Analysis)

PCA: 여러 변수 간에 존재하는 상관관계를 이용해 이를 대표하는 주성분을 추출해 차원을 축소하는 기법

- 데이터의 정보 유실 최소화하기 위해 높은 분산을 가지는 데이터의 축으로 차원 축소 → 주성분!
- 데이터의 변동성이 가장 큰 방향으로 축 생성 후 데이터를 투영



- 원본 데이터의 피쳐 개수에 비해 매우 작은 주성분으로 원본 데이터의 총 변동성을 대부분 설명 가능

선형대수 관점

- 입력 데이터의 공분산 행렬을 고유값 분해하고, 이렇게 구한 고유벡터에 입력 데이터를 선형 변환
- 고유벡터: PCA의 주성분 벡터, 입력 데이터의 분산이 큰 방향을 나타냄
행렬을 곱하더라도 방향이 변하지 않고 크기만 변함
- 고유값: 고유벡터의 크기, 입력 데이터의 분산
- 선형 변환: 특정 벡터에 행렬 A를 곱해 새로운 벡터로 변환하는 것
- 공분산: 두 변수 간의 데이터 변동
ex) $\text{Cov}(X, Y) > 0$: X가 증가할 때 Y도 증가
- **공분산 행렬**: 여러 변수와 관련된 공분산을 포함하는 정방형 행렬

	X	Y	Z
X	3.0	-0.71	-0.24
Y	-0.71	4.5	0.28
Z	-0.24	0.28	0.91

정방행렬 + 대칭행렬 → 항상 고유벡터를 직교행렬로, 고유값을 정방행렬로 대각화 가능

- **공분산 행렬 C**

$$C = P \Sigma P^T$$

P : $n \times n$ 의 직교 행렬

Σ : $n \times n$ 의 정방행렬

P^T : 행렬 P 의 전치 행렬

$$C = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$$

공분산 C = 고유벡터 직교 행렬 * 고유값 정방 행렬 * 고유벡터 직교 행렬의 전치 행렬로 분해

e_i : i 번째 고유벡터

λ_i : i 번째 고유벡터의 크기

e_1 : 가장 분산이 큰 방향을 가진 고유벡터

e_2 : e_1 에 수직이면서 다음으로 가장 분산이 큰 방향을 가진 고유벡터

- **PCA: 입력 데이터의 공분산 행렬이 고유벡터와 고유값으로 분해될 수 있으며, 이렇게 분해된 고유벡터를 이용해 입력 데이터를 선형 변환하는 방식**

1. 입력 데이터 세트의 공분산 행렬을 생성합니다.
2. 공분산 행렬의 고유벡터와 고유값을 계산합니다.
3. 고유값이 가장 큰 순으로 K 개(PCA 변환 차수만큼)만큼 고유벡터를 추출합니다.
4. 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환합니다.

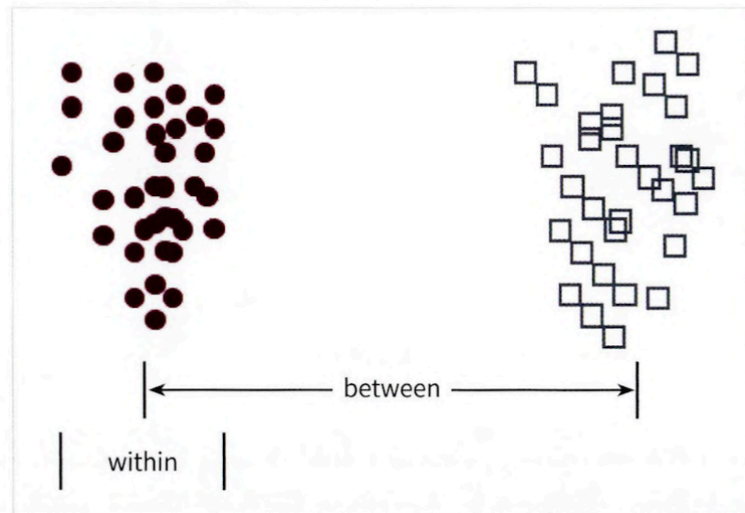
▼ 3. LDA (Linear Discriminant Analysis)

LDA: 선형 판별 분석법

지도학습의 분류에서 사용하기 쉽도록 개별 클래스를 분별할 수 있는 기준을 최대한 유지하면서 차원을 축소

- 입력 데이터의 결정 값 클래스를 최대한으로 분리할 수 있는 축을 찾음
- 클래스 간 분산과 클래스 내부 분산의 비율을 최대화

- 클래스 간 분산은 최대한 크게, 클래스 내부의 분산은 최대한 작게



- 클래스 간 분산과 클래스 내부 분산 행렬을 생성한 뒤, 이 행렬에 기반해 고유벡터를 구하고 입력 데이터를 투영
 - 클래스 내부와 클래스 간 분산 행렬을 구합니다. 이 두 개의 행렬은 입력 데이터의 결정 값 클래스별로 개별 피처의 평균 벡터(mean vector)를 기반으로 구합니다.
 - 클래스 내부 분산 행렬을 S_w , 클래스 간 분산 행렬을 S_b 라고 하면 다음 식으로 두 행렬을 고유벡터로 분해할 수 있습니다.

$$S_w^T S_b = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

- 고유값이 가장 큰 순으로 K개(LDA변환 차수만큼) 추출합니다.
- 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환합니다.

▼ 4. SVD (Singular Value Decomposition)

SVD: 특이값 분해

- $m \times n$ 크기의 행렬 A를 다음과 같이 분해

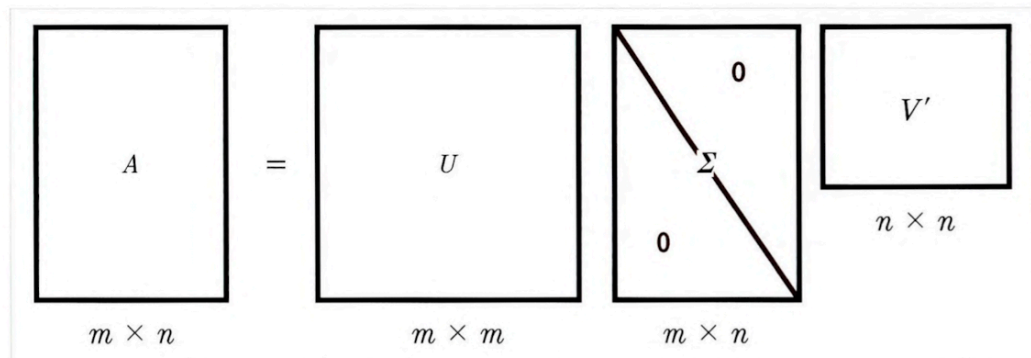
$$A = U \Sigma V^T$$

행렬 U와 V에 속한 벡터: 특이벡터 → 서로 직교

Σ : 대각행렬, 행렬의 대각에 위치한 값 외 나머지 위치의 값은 모두 0

Σ 이 위치한 0이 아닌 값 $\rightarrow$ 행렬 A 의 특이값

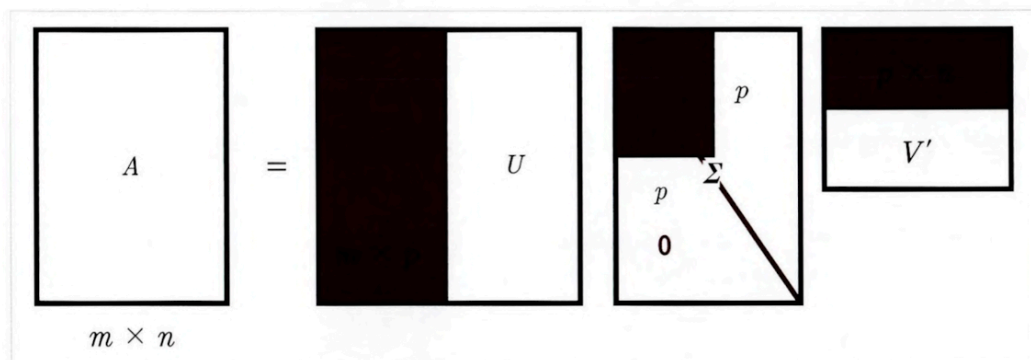
- **A의 차원이 $m \times n$ 일 때** $\rightarrow U: m \times m / \Sigma: m \times n / V^T: n \times n$



Σ 의 비대각인 부분과 대각원소 중에 특이값이 0인 부분도 모두 제거

제거된 Σ 에 대응되는 U 와 V 원소도 함께 제거

- **A의 차원이 $m \times n$ 일 때** $\rightarrow U: m \times p / \Sigma: p \times p / V^T: p \times n$



- **Truncated SVD**

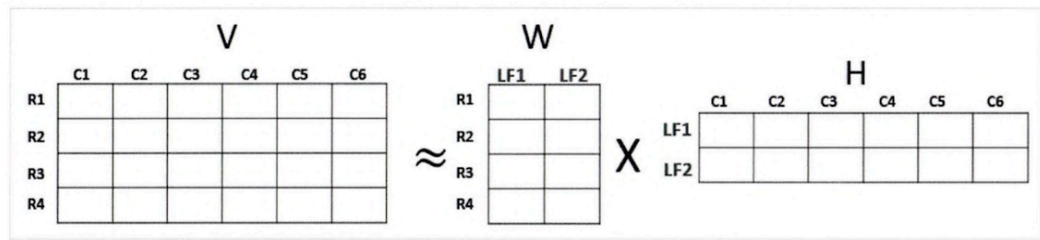
Σ 의 대각원소 중에 상위 몇 개만 추출해서 여기에 대응하는 U 와 V 의 원소도 함께 제거해 더욱 차원을 줄인 형태로 분해하는 것

인위적으로 더 작은 차원의 U, Σ, V^T 로 분해하기 때문에 원본 행렬을 정확하게 다시 원상복구할 수 없음

▼ 5. NMF (Non-Negative Matrix Factorization)

- **NMF:** 원본 행렬 내의 모든 원소 값이 모두 양수라는 게 보장되면 더 간단하게 두 개의 기반 양수 행렬로 분해될 수 있는 기법

Truncated SVD와 같이 낮은 랭크를 통한 행렬 근사 방식의 변형



- **행렬 분해:** SVD와 같은 행렬 분해 기법 통칭

길고 가는 행렬(W) & 작고 넓은 행렬(H)로 분해됨

잠재 요소

- W: 원본 행에 대해서 이 잠재 요소의 값이 얼마나 되는지에 대응
- H: 이 잠재 요소가 원본 열로 어떻게 구성됐는지를 나타냄